# Propuestas de mejora — Gestión Pro Total (Mantenimiento e Instalaciones)

Documento de trabajo con opciones de mejora priorizadas para la plataforma. Cada bloque indica **objetivo**, **impacto** y **esfuerzo** orientativo (B = bajo, M = medio, A = alto).

---

## 1. Calidad de código y mantenibilidad

| Mejora | Impacto | Esfuerzo |
|---|---|---|
| Activar `typecheck` en CI y resolver los errores de tipos (hoy `tsc` falla). Empezar por anotar los componentes de `src/components/ui`. | Menos bugs en producción | M |
| Añadir el job de `lint` al workflow de CI (`.github/workflows/node.js.yml`) para que no vuelvan a colarse imports/variables sin usar. | Evita regresiones | B |
| Tests automatizados: Vitest + React Testing Library para las calculadoras técnicas (Thermal, Solar, Electrical, ColdRoom) y utilidades de `src/utils`. | Alta — lógica crítica de negocio | M |
| Dividir páginas muy grandes (varias superan las 1.000 líneas) en subcomponentes y hooks reutilizables. | Mantenibilidad | M |
| Centralizar el acceso a datos (`base44.entities.*`) en hooks de React Query ya presente (`@tanstack/react-query`) para caché, reintentos y estados de carga homogéneos. | Rendimiento + UX | M |

## 2. Rendimiento

- **Code-splitting por ruta**: cargar cada página con `React.lazy` + `Suspense` para reducir el bundle inicial (hoy se cargan 51 páginas juntas).
- **Memoización** de listas y filtros pesados (Contacts, Inventory, Documents, Reports ya iniciados) — extender el patrón `useMemo` / `useCallback`.
- **Virtualización** de tablas largas (inventario, contactos, facturas) con `@tanstack/react-virtual`.
- **Optimización de imágenes** de galerías de reparación (lazy loading + thumbnails).

## 3. Experiencia de usuario (UX)

- Modo **offline** real: con la PWA + Capacitor, cachear datos para técnicos en campo sin cobertura (ya hay `TechnicianFieldWork`).
- **Atajos de teclado** y paleta de comandos (ya está `cmdk` en dependencias).
- **Estados vacíos** y *skeletons* consistentes en todas las páginas.
- **Accesibilidad** (a11y): foco visible, roles ARIA, contraste — auditar con axe.
- **Internacionalización** (i18n) si se prevé uso fuera de España.

## 4. Seguridad

- **Variables de entorno**: documentar y validar en arranque que `VITE_BASE44_APP_ID` y `VITE_BASE44_APP_BASE_URL` existen; fallar con mensaje claro.
- **Dependencias**: `npm audit` reporta vulnerabilidades (9 moderadas, 11 altas, 1 crítica). Planificar actualización por lotes y fijar versiones.
- **Roles y permisos**: revisar `requiresAuth: false` en `base44Client.js`; activar autenticación donde el portal de cliente y datos sensibles lo requieran.
- **Registro de auditoría**: ya existe `AuditLog` / `ChangeLog`; asegurar que cubre acciones críticas (borrados, cambios de precios, firmas).

## 5. Funcionalidad de negocio

- **Firma digital** avanzada y sellado de tiempo en documentos legales (`LegalDocs`, `PendingSignaturesPanel`).
- **Facturación electrónica** conforme a normativa española (Verifactu / TicketBAI según comunidad). Alto valor para un SAT.
- **Integración con pasarela de pagos** (ya hay Stripe en dependencias) para cobros online desde el portal de cliente.
- **Geolocalización y rutas** de técnicos (ya hay `react-leaflet` y `TecniciansRoutes`): optimización de rutas y *tracking* en tiempo real.
- **Notificaciones push** (móvil con Capacitor) para avisos de SAT y recordatorios.
- **Exportación contable** a formatos de gestorías (CSV/Excel ya con `xlsx`).

## 6. Datos e Inteligencia

- Consolidar los módulos de IA (`AIAssistant`, `AIBudget`, `AIReports`, `AIPlanning`) bajo un cliente común y *prompts* versionados.
- **Cuadro de mando** (KPIs) con previsión de ingresos, carga de trabajo y stock crítico (ya hay `recharts`).
- **Mantenimiento predictivo** a partir del histórico de averías (`EquipmentFailureHistory`).

## 7. DevOps y entrega

- **CI completo**: lint + typecheck + test + build en cada PR.
- **Despliegue automatizado** (preview por PR) y *releases* versionadas.
- **Build multiplataforma** (ver `docs/BUILD-MULTIPLATAFORMA.md`): APK Android, escritorio (Electron) y PWA, todo desde el mismo código.
- **Gestión de secretos** en GitHub Actions para los identificadores de Base44.

---

## Priorización sugerida (primeros 30 días)

1. Lint + typecheck en CI (B) — base de calidad.
2. Validación de variables de entorno y `npm audit` crítico (B/M) — seguridad.
3. Code-splitting por ruta (M) — rendimiento percibido.
4. Tests de las calculadoras técnicas (M) — protege la lógica de negocio.
5. Empaquetado APK vía CI (M) — entrega móvil (ya preparado en este repositorio).